

型付き AI 記述言語 AIPL の設計と実装

Yasushi Kodama

kodamay@

2026-05-13

Abstract

本稿はアクター指向の並列・並行言語 **AIPL** (Actor-based Intelligent Parallel Language; 旧称 ABCL/c+) の設計と実装を, **Hindley-Milner 型推論** を中核とする型システムを軸に記述する. AIPL は同一ソースに対し **7 種のランタイム実装** と **8 種のコード生成ターゲット** を持つが, このうち推論を採用するのは Python (推論版), OCaml, そして C コード生成 (aipl2c の HM + コード特殊化経路) の 3 実装である. 本研究の貢献は四つある. 第一に, アクター言語の表面構文に注釈を一切要求せず, フィールド型・メソッドシグネチャを呼び出しサイトと初期化式から自動推論する **型注釈フリーなアクター言語** を実現し, 3 実装間で **同一の推論結果** を生成することを経験的に検証した. 第二に, 推論結果を活用してアクター間メッセージ境界以外の領域を **value_t** タグ付き共用体から **ネイティブ C 型に特殊化** する HM 駆動コード生成を確立した. 第三に, 1986 年の ABCL/1 系譜のアクターモデルに現代的な LLM 連携を組み込み, **ai_call(provider, prompt)** プリミティブとして第一級化した. 第四に, 同一の **.abcl** ソースから OS スレッド (1:1) / 軽量スレッド (M:N) / 協調イベントループの 3 並行モデルすべてに展開できる **ターゲット非依存セマンティクス** を確立し, AIPL を AIPL で記述する完全な **セルフホスト塔** (Level A-C-3, 37/37 スモークテスト合格) で言語仕様の自己一貫性を実証した. 段階的型注釈に基づく旧来の Python 実装 (Python 注釈版) は, より広範な研究機能 (Phase 11-17, 線形型/効果系/所有/セッション型) を抱える参照実装として残置し, 推論版との対比で型システム研究の二つの軸を一つの言語で同時に比較できるようにした.

Contents

1 はじめに	1
2 背景	2
2.1 アクターモデルと ABCL/1 系譜	2
2.2 現代の型システムの蓄積	2
2.3 LLM 連携の必要性	2
3 言語設計	2
3.1 コア構文	2
3.2 型システム — Hindley-Milner 推論を中核に	3
3.2.1 HM 推論 (本稿の中心)	3
3.2.2 HM 推論の 3 実装	4
3.2.3 推論駆動のコード特殊化 (C 経路)	4
3.2.4 オプショナルな上位機能 (Phase 11-17)	4
3.3 LLM 統合	5
3.3.1 ai_call API	5
3.3.2 3 種の送信形式から LLM を呼ぶ	5
3.4 3 並行モデルの統一	7
4 実装	7
4.1 7 ランタイム	7
4.2 8 コード生成ターゲット	8
4.3 コード生成の中核技術	8

5	セルフホスト塔	9
6	評価	9
6.1	サンプル網羅性	9
6.2	スモークテスト結果 (2026-05-13 最新)	10
6.3	型推論のクロス検証 (主要結果)	10
6.4	LLM 統合の実証	11
6.5	WebSocket 統合	11
7	関連研究	11
8	結論と今後の課題	12

1 はじめに

並行計算と人工知能の境界はかつてないほど混ざりつつある。大規模言語モデル (LLM) を中心とする AI システムは、本質的に複数のエージェントが協調するアーキテクチャに収束しつつあり、これは 1980 年代の Hewitt のアクターモデル [1] と Yonezawa らの ABCL/1 [2] が描いた未来像と驚くほど近い。しかし当時の言語設計には LLM や型推論・線形性・セッション型など、その後 30 年の研究成果が当然ながら含まれていなかった。

本研究は、アクター言語の系譜を母艦としつつ現代の言語理論の成果を重ねる試みとして **AIPL** (Actor-based Intelligent Parallel Language) を提案する。特に本稿は AIPL の**型推論版** (Python 推論版, OCaml, C コード生成の HM 経路) を中心に据え、これら 3 実装が同一ソースに対して**同一の推論結果**を生成することを研究の主軸とする。AIPL は次の四つを同時に達成することを目標とする:

1. **型注釈フリーなアクター記述**: Hindley-Milner 推論を中核とし、フィールド型・メソッドシグネチャをリテラル・初期化式・呼び出しサイトから自動推論する。プログラマは `int` などの注釈を書く必要がない。
2. **推論駆動のコード特殊化**: 推論型を C コード生成器に渡し、アクター間メッセージ境界以外を `value_t` タグ付き共用体から `long/double/const char*` 等のネイティブ C 型に特殊化する。
3. **アクター意味論の保存**: `send/now/future/await` の四プリミティブで Hewitt 型アクター [1] を記述でき、LLM 連携 `ai_call` を組み込みとして第一級化する。
4. **実装多様性と意味論保存**: 同一の `.abcl` ソースを 7 種のランタイムと 8 種のコード生成ターゲットで実行でき、HM ベースの 3 実装は推論結果が一致する。

段階的型注釈に基づく旧来の Python 実装 (Python 注釈版) は、Phase 11–17 の高度型機能 (capability 効果系, CSP チャネル, 線形型, 所有型, 構造化並行, `transient cast`, セッション型) を含む参照実装として位置付け、本稿でも適宜参照する。推論版と注釈版を一つの言語仕様に対する二つの実装として比較できる構造は、本研究固有の利点である。

論文の構成は次の通りである。§2 で背景を、§3 で言語設計を (§3.2 で型推論を中心に論じる)、§4 で 7 ランタイム + 8 コード生成ターゲットの実装を述べる。§5 でセルフホスト塔、§6 で評価結果 (§6.3 の推論クロス検証が中心)、§7 で関連研究、§8 で結論と今後の課題を示す。

2 背景

2.1 アクターモデルと ABCL/1 系譜

Hewitt らの提案したアクターモデル [1] は、計算を **独立に状態を持つアクター間の非同期メッセージ送信**として定式化する。AIPL の直接の先祖である ABCL/1 [2] は、これに次の三種類の送信を導入した:

- *past type* (`send`) — fire-and-forget の非同期送信.
- *now type* (`now`) — 同期送信, 呼び出し側を返答到着までブロック.
- *future type* (`future`) — 非同期送信, 後で `await` で値取得.

これら三種は Erlang ($\rightarrow !$ と `gen_server:call`) や Akka ($\rightarrow \text{tell} / \text{ask}$), Pony の `behaviours` など, 後続のアクター言語に直接または間接に継承されている.

2.2 現代の型システムの蓄積

2010 年代以降, 並行プログラムの安全性を高めるための型理論的成果が複数得られている:

- **capability effects** [4]: 関数の副作用を型レベルで追跡 (`!{fs, ai, net, mut}` 形式).
- **linear / affine types** [5]: 値の使用回数を型レベルで制限 (`use-after-move` を静的に検出).
- **ownership / capabilities** [6]: フィールドのアクセス権を型階層に組込む (Rust や Pony で実用化).
- **session types** [7]: プロトコル全体を一つの型として表現, 通信順序の不正を静的または動的に検出.

AIPL の Phase 11-17 はこれらを順次取り入れる.

2.3 LLM 連携の必要性

LLM 駆動の AI システムは, 複数のエージェント (`planner / solver / reviewer` など) が非同期に協調する典型例である. これを既存の汎用言語で書くと, スレッド管理・タイムアウト・予算管理・フォールバックといった雑事のコードが本来のドメインロジックを覆い隠してしまう. AIPL は `ai_call` を組み込みとして第一級化し, これらを言語仕様の側で吸収することを試みる.

3 言語設計

3.1 コア構文

AIPL のコア構文を最小限のクラス例で示す:

```
class Counter {
  var count = 0;
  method tick() {
    count = count + 1;
    print("count = " + count);
  }
  method get() {
    reply(count);
  }
}

var c = new Counter();
send c.tick();           // past — fire-and-forget
var v = now c.get();      // now — block, return reply
var f = future c.get();   // future — handle to async reply
var w = await(f);         // ... pick up the future later
```

主な構造的要素:

- **class 宣言**. 各クラスはアクターのテンプレートで, フィールド (`var`) とメソッド (`method`) を持つ.
- **init メソッド**が `new C(args)` で自動的に呼ばれる.

- メッセージ送信は `send / now / future` の三種.
- `await(f)` で `future` を解決, `reply(v)` で同期送信に応答.
- `become C(args)` でアクターのクラスを実行時に差し替え.
- `select` で複数チャネルからの選択受信 (Phase 13).

3.2 型システム — Hindley-Milner 推論を中核に

AIPL の型システムは **HM 推論** を中核に置き, その周囲にオプションな注釈ベースの上位機能 (Phase 11–17) を配する設計である. 本稿が主に論じるのは前者である.

3.2.1 HM 推論 (本稿の中心)

AIPL の `.abcl` ソースには通常, 型注釈を一切書かない:

```
class Hello {
  var count = 0.;           // 注釈なし — 0. リテラルから float に推論
  method init(n) {          // n も注釈なし
    count = n;              // count : float なので n : float に
    print("hi " + n);
  }
  method greet() {
    print("count = " + count);
  }
}
var h = new Hello(5);      // 呼び出しサイトから init の引数を再特殊化
```

推論アルゴリズム OCaml の `infer.ml` (484 行) は教科書通りの Hindley-Milner を実装する: `unify / generalize / instantiate`, mutable union-find な型変数, `let`-多相化を備える. クラスメソッドはまず仮の型変数で事前登録し, 本体検査時にボトムアップで具体化する 2 パス方式である.

call-site による自動モノモルフィゼーション クラスメソッドが多相型のままでも, 呼び出しサイトの引数型から自動的に具体化される. 例えば `Counter.dec(x)` は宣言時には $\forall \alpha. \alpha \rightarrow \text{unit}$ だが, 本体内で `count - x` (`count : float`) によって $\alpha = \text{float}$ に固定される. 別途モノモルフィゼーション・パスは不要で, HM の `unify` がこの役割を自動で果たす.

センチネルパターンの widening `var partner = 0;` と整数初期化したフィールドに後でアクター参照を代入する古典的な actor-language 慣用句では純粋な HM は失敗する. 我々は flow-sensitive な **join 拡張** (monotonic widening) を補助的に組み合わせ, Python 推論版・browser-abcl で実装した. 純 HM 派の OCaml / C 経路ではこのパターンは any-型へ後退するかコンパイル時の不一致として顕在化する.

3.2.2 HM 推論の 3 実装

本稿の主要な実装の貢献は, 以下の 3 実装が同一アルゴリズムをそれぞれの言語で再実装し, 同じ結果を出すことを経験的に検証した点にある (詳細は §6.3):

- **OCaml 実装** (`src/infer.ml + types.ml + typing_env.ml`, 合計 ~985 行)
- **Python 推論版** (`src/python-aipl-inferred/aipl_infer.py`, 899 行) — OCaml の HM を Python に直接移植
- **C コード生成** (`src/aipl2c.ml + c_translator.ml`) — OCaml の推論結果を `Types.class_field_types` と `Types.class_method_schemes` レジストリ経由で取得し, C コードに型情報を直接埋め込む

これに加え, JS 系の 2 実装 (browser-abcl, node-aipl-server) は **flow-sensitive な軽量推論** (308 行の `typecheck.js`) を採用する. HM とは異なるアルゴリズムだが基本的なフィールド型推論には十分機能する.

3.2.3 推論駆動のコード特殊化 (C 経路)

HM 推論の最大の実用的成果はコード生成側にある. `c_translator.ml` は推論結果を活用し, アクター内部の式・代入・ローカル変数を `value_t` タグ付き共用体ではなく **ネイティブ C 型** で発行する:

```
// 注釈ベース時代 (推論なし)
value_t l_x = v_binop("+", l_a, l_b);

// HM 推論ベース (l_a, l_b が int と推論される場合)
long l_x = (l_a + l_b);
```

`v_binop` のディスパッチを経由しない**ネイティブ加算**が発行されるため, ホットパスの実行速度が大きく改善する. アクター間メッセージ境界 (`value_t args[]`) は `uniform` であり続けるため, mailbox の `uniformity` と内部の特殊化を両立できる.

3.2.4 オプションな上位機能 (Phase 11–17)

注釈ベースの研究機能は Python 注釈版が参照実装として保持する. 本稿で扱う推論版では, これらは原則として**ランタイム側**のみに存在し, 静的検査は HM 推論で代替される (例えば line 14/15/16 の特性は実行時にしか問わない).

Phase 12: capability 効果 `!fs, ai, net, mut` で副作用集合を宣言. 推論版はランタイムを共有するため動的に動くが, 静的検査は注釈版のみが行う.

Phase 13: CSP チャネル `channel[T]` 型, `channel_send / channel_recv / select_recv` を Python ランタイムに実装. 両 Python 版で動作する.

Phase 14: 線形型, Phase 15: 所有 (pub/owned) 注釈ベース. 推論版では型レベル `enforcement` は無いが, ランタイムは共通なので実行は可能.

Phase 16: transient cast, Phase 17: 構造化並行 それぞれ `any` 境界での実行時型キャストと `scope { future ... }` の自動 `join`. 後者は推論版にもランタイム経由で利用可能 (Java の Project Loom [11] と意味論的に同型).

セッション型 (動的) プロトコル仕様を文字列で宣言し, 通信を実行時に検証する:

```
session_define("Solve",
  "solver.solve(string) ! string"
  " -> reviewer.review(string,string) ! string"
  " -> planner.solve(string) ! string");
```

HM 推論と独立した**実行時動的検査**として位置付けるため, 推論版にも自然に組込まれる.

3.3 LLM 統合

3.3.1 ai_call API

`ai_call` は同期的に LLM プロバイダを呼び出して文字列を返すプリミティブである:

```
var r1 = ai_call("prompt");           // env 自動選択 (default: Gemini)
var r2 = ai_call(1, "prompt");         // 1 = Gemini
var r3 = ai_call(2, "prompt");         // 2 = Anthropic (Claude)
var r4 = ai_call(3, "prompt");         // 3 = OpenAI (ChatGPT)
var r5 = ai_call_with_system("sys", "p"); // システムプロンプト付き
var r6 = ai_call_retry(3, "prompt");   // 自動リトライ
```

ランタイムは以下を提供する:

- トークン予算管理 (ABCL_AI_TOKEN_BUDGET, ai_remaining()).
- 同時実行上限 (ABCL_AI_MAX_CONCURRENT).
- フォールバック連鎖 (ABCL_AI_FALLBACK_MODELS).
- テスト用 mock プロバイダ (ABCL_AI_PROVIDER=mock で API 鍵不要, オフライン smoke 用).

3.3.2 3 種の送信形式から LLM を呼ぶ

ai_call 自体は同期的だが, アクターのメソッド本体から呼ぶことで, アクター言語の三送信形式 (past/now/future) すべてに自然に組み込める. 以下では「LLM を実際に呼ぶ Asker アクター」を共通基盤として, それを past / now / future の三形式で駆動する例を示す.

共通の Asker アクター. どの形式でも, LLM 呼び出し自体は同じアクターメソッドに閉じる:

```
class Asker {
  method ask(q) {
    var r = ai_call(2, q);    // 2 = Anthropic (Claude)
    reply(r);                // 同期送信に応答 (now / future が拾う)
  }
}
```

ここでメソッド内の reply(r) は, 呼び出し側が now または future で呼んだ場合の返答スロットへ値を渡す.send で呼ばれた場合 reply は捨てられる (fire-and-forget の意味論).

(1) now —同期送信: ブロックして LLM 応答を待つ.

```
var a = new Asker();
var answer = now a.ask("In 5 words, why is the sky blue?");
print(answer);    // → "Rayleigh scatters short blue wavelengths."
```

呼び出し側のアクター (あるいは最上位) が now で発射すると, Asker.ask の本体が走り終えて reply(r) に到達するまでブロックする. 内部的にはランタイムが reply スロットを発行し, Asker スレッドが reply を呼んだ時点でその slot を fulfill して待機側を起こす. 外部から見ると, ai_call と now の合成は通常の同期関数呼び出しと同じ意味論で振る舞う.

(2) future + await —非同期発射 + 後で値取得.

```
var a = new Asker();
var f = future a.ask("Summarise quicksort in one sentence.");
// ... ここで f の到着を待たず別の仕事をする ...
do_other_work();
var answer = await(f);
print(answer);
```

future は Asker.ask の終了を待たずにハンドル f を即座に返す. 呼び出し側はその間に他の計算 (例えば別の ai_call の並列発射) を進められる.await(f) で値を要求した時点で reply スロットが未到着なら, そこで初めてブロックする.AI コールを並列実行して結果を後で合流する典型的なパターンに対応:

```
var a = new Asker();
var f1 = future a.ask("Translate hello to French.");
var f2 = future a.ask("Translate hello to German.");
var f3 = future a.ask("Translate hello to Japanese.");
print(await(f1));    // 三本の AI コールが並列に走る
print(await(f2));    // (ランタイムが同時実行上限内で多重化)
print(await(f3));
```


(3) **send (past type) + callback** —完全非同期. send は fire-and-forget で reply を持たないため, 結果は コールバック先のアクターを引数で渡して受け取らせる:

```
class Asker {
  method ask(q, callback) {
    var r = ai_call(2, q);
    send callback.got(r);    // コールバック先へ非同期で結果を流す
  }
}
class Printer {
  method got(r) { print("[got] " + r); }
}
var a = new Asker();
var p = new Printer();
send a.ask("Why is the sea salty?", p);    // ここでは値は待たない
```

このパターンは典型的なメッセージ・パッシングのスタイルである.LLM 応答を受け取るアクターが固有の責務 (例えば応答の検証・記録・さらなる派生呼び出し) を持つ場合に向いている.

形式選択の指針. 三送信形式の使い分けは, アクター言語一般の指針と同じで, LLM 呼び出しに固有の話ではない:

形式	選択基準
now	値が直後に必要で, 並列化の余地がない. 制御フローが線形で読みやすい.
future	複数 LLM コールを並列発射し, 後で合流したい. 遅延応答に対するスループット重視.
send	応答の処理が別アクターの責務として独立. ストリーミング処理やイベント駆動的なパイプライン.

実装間の保持. 重要なのは, この三形式 × LLM の組合せが全 7 ランタイム実装 (Py-A,Py-I,OCaml,JS-O,JS-B,JS-N,C のうち AI 統合があるもの) で **同じ表面構文・同じ意味論** で動作する点である (§6 の評価表 6.4 参照). プログラマは ai_call の周囲に AIPL 標準のアクター送受信を組合せるだけで,LLM 連携アプリケーションをランタイム非依存に書ける.

3.4 3 並行モデルの統一

AIPL のセマンティクスは, 以下の異なる並行モデル上で同型に実行できるように設計されている:

1. **1:1 OS スレッド層:** 各アクターが独立した kernel-scheduled なスレッドとして走る.Python の threading.Thread, OCaml の Thread.create,C の pthread_create が該当. ブロッキング I/O が他アクターを止めない代わりに,kernel スケジュールのオーバーヘッドにより ~1000 アクター程度がスケール上限.
2. **M:N 軽量スレッド層:** ランタイムが小さな OS スレッドプール上でアクターを多重化する.Pony actor / Erlang BEAM process / Go goroutine が該当.~ 10^5 – 10^7 アクターまで実用的にスケール.
3. **協調イベントループ層:** 単一 OS スレッドで setTimeout(0) ベースに mailbox を順次処理. browser-abcl と node-aipl-server が該当. 並行性はないがメッセージ間のインターリーブは保たれ, ブラウザ DOM に統合できる.

同一の .abcl ファイルが, コード変更なくこの三層のいずれにも配置可能であることが本研究の重要な実証成果である.

4 実装

AIPL は単一言語に対して 7 種のランタイム実装と 8 種のコード生成ターゲットを持つ。ランタイムはソースコードをパースしてそのまま解釈実行する。コード生成ターゲットは aipl2c コマンドで他言語のソースコードに変換する経路である。

4.1 7 ランタイム

本研究の中核となる HM 推論版 3 実装を最初に置き、その後ろに注釈版と JS 系の flow-sensitive 推論を配する:

短縮名	ソース	型システム	特徴
<i>HM 推論版 (本稿の中心)</i>			
OCaml	src/*.ml	HM 推論	正典実装.infer.ml 484 行.web_gateway に WebSocket 内蔵
Python (推論)	src/python-aipl-infer	HM 推論	OCaml の infer.ml を Python に直接移植 (899 行)
C (aipl2c)	src/aipl2c.ml + 各種 C runtime	HM + コード特殊化	推論結果を C 型に直接埋込.value_t 不要な経路を生成
<i>Flow-sensitive 推論版 (JS 系)</i>			
JS-Browser	src/browser-abcl/	flow-sensitive (typecheck.js)	ブラウザ単体,canvas デモ可
JS-Node	src/node-aipl-server/	flow-sensitive (継承)	Node HTTP server./api/typecheck + /ws
JS-OCaml	src/app.js 等 + OCaml gateway	(= OCaml HM)	ブラウザ JS が HTTP で OCaml バックエンドを駆動
<i>注釈版 (参照実装)</i>			
Python (注釈)	src/python-aipl/	段階的型注釈 (gradual)	Phase 11-17 + 動的 compile + メソッドインジェクションを保持する研究用参照

4.2 8 コード生成ターゲット

aipl2c は OCaml で書かれたコード生成器で、フラグにより出力言語を切り替える:

フラグ	ターゲット	アクター単位	用途
(なし)	C + pthread	1:1 OS スレッド	軽量スタンドアロン
(-lSDL2)	C + SDL2 GUI	同上 + GUI スレッド	可視化デモ
--xinu	Xinu C	1:1 OS プロセス	組込み OS
--python	Python	1:1 OS スレッド	配布用ピュア Python
--pony	Pony	M:N (Pony 作業窃取)	capability-secure な研究用
--erlang	Erlang	M:N (BEAM)	軽量プロセス,hot reload
--go	Go	M:N (goroutine)	プロダクション展開
--prolog	SWI-Prolog	1:1 OS スレッド (SWI)	論理プログラミング統合

すべて単一のソースを共有する.Hello.abcl を例に取れば,8 ターゲットすべてで意味的に同じ出力 (Hello! count = 5 等) を得る。

4.3 コード生成の中核技術

HM 推論結果に基づくコード特殊化 (C ターゲット) `src/c_translator.ml` は推論済みの型をクラスフィールド・メソッドパラメータ・ローカル変数の C 宣言に直接反映する:

```
// AIPL: class Hello { float count = 0.; method init(n) {...} }
// 生成 C:
static void Hello_init(int self_id, int sender_id,
                      value_t* args, int n_args) {
    long p_n = (n_args > 0)
        ? (args[0].tag == V_INT ? args[0].i : (long)args[0].f)
        : 0L;
    objects[self_id].fields[F_Hello_count] = mk_float((double)p_n);
    // ...
}
```

`value_t` タグ付き共用体はアクター間メッセージ境界のみで使用し、メソッド本体内部はネイティブ C 型に特殊化される。算術演算は `v_binop()` デイスポッチを経由せず直接 `(double)a + (double)b` を発行するため、ホットパスでのオーバーヘッドが消える。

ターゲット言語のアクター意味論への射影 8 ターゲットそれぞれが、AIPL の `send obj.method(args)` 構文を異なる仕組みに対応付けている:

ターゲット	AIPL <code>send</code> の射影
-------	----------------------------

C+pthread	<code>enqueue(self_id, obj_id, "method", args)</code>
Python	<code>obj._enqueue("method", args)</code>
Pony	<code>obj.method(args)</code> (Pony の actor は <code>async</code> デフォルト)
Erlang	<code>Pid ! {method, Args}</code>
Go	<code>obj.Method(args)</code> 経由で <code>obj.mailbox <- msgT{...}</code>
SWI-Prolog	<code>thread_send_message(Tid, method(Args))</code>

二段階初期化 (Pony / Go / Erlang) AIPL の `new C(args)` は構築と `init` 呼び出しを暗黙に合成するが、Pony や Go の strict なコンストラクタ意味論では、構築時にクロスアクター参照が未確定でも安全に `init` を起動する必要がある。これを次の二段階パターンで解決した:

```
// AIPL:
var p = new Pinger();
var q = new Ponger();
send p.bind(q);

// Go 出力 (抜粋):
p := NewPinger()           // 構築のみ, init body はまだ走らない
q := NewPonger()
p.Init()                   // _aipl_init behaviour に init body を委譲
q.Init()
p.Bind(q)                  // クロスアクター参照が確実に存在する状態で実行
```

5 セルフホスト塔

`aipl-self-host/` 配下に、AIPL を AIPL 自身で記述した 9 段階の塔がある。Python ランタイムをホストとして AIPL コードが AIPL コードを評価する構造:

Level	実装内容	何が動くか	Sm.
A	metacircular eval(ast, env)	Arith, Control, Fib, Hello	4/4
B-1	Phase 11 type checker	6 sample	6/6
B-2	Phase 12 capability effects	4 sample	4/4
B-3	Phase 13 channel element types	5 sample	5/5
B-4	Phase 14 linear / use-after-move	4 sample	4/4
B-5	Phase 15 owned / pub visibility	4 sample	4/4
C	lexer + parser + eval パイプライン	AIPL が AIPL を読む	3/3
C-2	actor scheduler + mailbox	6 sample	6/6
C-3	now / Reply 同期送信	1 sample	1/1
合計		全 9 段階	37/37

Phase monotonicity の自動証明. 各 Level B-N の checker は, それ自身の .abcl ソースに対しても自分のルールが矛盾なく適用できることを SampleSelfConsistency.abcl で確認している. これは「Phase N の checker は Phase N の規約に従って書かれている」という不変条件の経験的検証となる.

6 評価

6.1 サンプル網羅性

ランタイムごとに到達可能なサンプル数 (core + AI 統合 + remote-actor):

カテゴリ	Py-A	Py-I	OCaml	JS-O	JS-B	JS-N	C
core	33	33	57	57	5	5	57
AI 統合	11	11	8	8	0	0	8
remote-actor	8	8	1	1	0	0	1
合計	52	52	66	66	5	5	66

クロスカットとして aipl-self-host が 48 個, docker/cross の言語間相互運用サンプルが 4 個ある.

6.2 スモークテスト結果 (2026-05-13 最新)

対象	PASS	xfail	FAIL
OCaml smoke (run_all_smoke_tests.sh)	48	0	9*
browser-abcl (syntax / parse / typecheck)	7+4+4	0	0
node-aipl-server (HTTP + WS)	10	0	0
python-aipl-inferred	20	0	0
Pony codegen	2	1	0
Erlang codegen	2	1	0
Go codegen	2	1	0
Prolog codegen	2	1	0
C + WebSocket (libwebsockets)	1	0	0
aipl-self-host (全 9 Level)	37	0	0

* OCaml smoke の 9 fail は abclc/ の Gui/Py/Xinu サンプル固有の既存型問題 (var partner = 0; を後で actor 参照で上書きするパターン). 本実装の追加変更による回帰ではない.

6.3 型推論のクロス検証 (主要結果)

本研究の中心的な経験結果として, 3 つの HM 実装 (OCaml / Python 推論版 / C コード生成) が, 同一の .abcl 入力に対して完全に一致する推論型を出力することを確認した:

```
// abclc/Hello.abcl
class Hello { float count = 0.; method init(n) { count = n; } method greet() {...} }
var h = new Hello(5);
```

実装	推論結果
OCaml	count:float, init:(int)->unit, greet:()->unit
Python-inferred	count:float, init:(int)->unit, greet:()->unit
C (aipl2c)	count:float, init:(int)->unit, greet:()->unit

3 実装すべてで init の引数型が呼び出しサイト (new Hello(5)) の整数リテラルから int に 自動モノモルフィゼーションされ, count のフィールド型が初期値 0. から float に固定される.HM 推論は実装言語に依存しない普遍的なアルゴリズムであることを, この一致が経験的に示している.

この結果の意義は二点ある. 第一に, プログラマは AIPL ソースに注釈を一切書かずに, 3 実装間で型の解釈が一致するという保証を得る. 第二に, C コード生成において HM 結果がそのままネイティブ C 型の選択に使えるため, 推論なしで生成されるタグ付き共用体ベースの C コードに比べ実行速度・コードサイズの両面で利得を得る.

注釈版が定義する Phase 11-17 の高度機能 (capability 効果系, 線形型, 所有, セッション型) は推論版にはランタイム経由でのみ存在する. このトレードオフを許容することで, 推論版は「型注釈フリー + ランタイム共有」という研究的に興味深いニッチを開いた.

6.4 LLM 統合の実証

ai_call の三送信パターン全てを 5 ランタイム (Python-A, Python-I, OCaml, JS-Browser, JS-Node) で確認した:

パターン	Py-A	Py-I	OCaml	JS-B	JS-N
now	✓	✓	✓	✓	✓
future + await	✓	✓	✓	✓	✓
send + callback	✓	✓	✓	✓	✓

OCaml では検査中に二つのバグを発見・修正した: (a) ABCL_AI_PROVIDER=mock が明示的な provider 引数を上書きすべき箇所での優先順位エラー, (b) 最上位 send a.ask("hello", rcv) が引数を評価しないまま配送する pre-existing バグ.

6.5 WebSocket 統合

全 7 ランタイムで WebSocket サーバを動かせる状態にした:

実装	経路
OCaml	web_gateway.ml に内蔵 (RFC 6455 自前実装, 40 LOC)
JS-OCaml	OCaml バックエンド経由
Python (annot/inf)	aipl_websocket.py (websockets ライブラリ)
JS-Browser	ブラウザネイティブ WebSocket API
JS-Node	ws ライブラリ, /ws?sid= + /api/broadcast
C (aipl2c)	abcl_ws_runtime.c (libwebsockets)

ワイヤープロトコルは全実装で統一: ?sid=<group> クエリでブロードキャストグループに参加, 接続時に {"kind":"welcome", "sid":"...", "peers":N} を受信, 以後同一 sid の peers にメッセージが転送される.

7 関連研究

ABCL/1 [2] 本研究の直接の先祖.AIPL は ABCL/1 の三種送信を継承し、現代的な型システムと LLM 統合を上重ねた。

Erlang/OTP [8] BEAM 仮想機械上の軽量プロセスとメッセージ受信パターンマッチング. AIPL の `--erlang` ターゲットはこの語彙で出力するため、意味論的に最も近い。

Akka [9] Scala の上のアクターライブラリ `.tell` (= AIPL `send`) と `ask` (= AIPL `now`) の対応がある.AIPL はライブラリではなく言語仕様レベルで同じセマンティクスを提供する。

Pony [10] Reference capabilities (`iso` / `val` / `ref` / `box` / `tag`) でアクター間のデータ共有を静的に検査する.AIPL の Phase 14 (`linear`) と Phase 15 (`owned`) はこの方向への近似であり,`--pony` ターゲットは両者の概念的対応を確認する自然な経路となる。

Project Loom (Java) [11] Java 21+ の `virtual threads`.AIPL の Phase 17 (`scope { future ... }`) は Loom の `StructuredTaskScope` と意味論がほぼ同型。

Session types [7] 以降の研究.AIPL のセッション型は完全に動的検査側に置いたため、複雑なプロトコルでも記述しやすい一方で静的な健全性保証は犠牲になっている。この設計選択は AIPL の研究的立ち位置 (実用主義) を反映している。

8 結論と今後の課題

本稿では、アクター言語 AIPL を 7 ランタイム + 8 コード生成ターゲットの **一言語多実装**として設計・実装した経過を報告した。主な貢献は:

1. LLM 連携を第一級プリミティブ `ai_call` として組み込み、プロバイダ・予算・並行性をランタイムが管理する設計。
2. Hindley-Milner 推論を中心とする多層型システム。実装ごとにどの層を有効にするかが独立に選べる。
3. 同一の `.abc1` を OS スレッド層・M:N 層・協調イベントループ層の三並行モデルすべてに展開できることの検証。
4. AIPL を AIPL で記述した 9 段階のセルフホスト塔 (`aipl-self-host/`) で言語仕様の自己一貫性を経験的に示した。

今後の課題

- **JVM 系コード生成** (Kotlin / Scala): Loom の `virtual threads` を活用すれば,JVM エコシステムへの自然な橋渡しとなる。
- **Swift コード生成**: Swift 5.5+ の `actor` 言語機能は AIPL の Phase 17 `scope` と意味論が同型で,Apple エコシステムへの到達範囲が広がる。
- **WebAssembly ターゲット**: 既存の C ターゲットを経由せず直接 WASM を出力すれば,browser / edge / serverless へのデプロイが軽量化される。
- **C ターゲットの機能補完**: 現状 `become` / `select` / `now` の `reply slot` / `ai_call` が C ランタイムに未実装.libcurl + 拡張 mailbox レイヤの追加で対応可能。
- **セルフホストの他ターゲットへの展開**: 現在 Python ランタイム上でのみ動く Level A-C-3 を,`--pony` や `--go` 出力上でも動かす実験は、セマンティクス保存の強い証拠となる。

- **Phase 18 予測:** aice-evolution-v2 の MAP-Elites GA を Phase 17 状態の AIPL に対して走らせ、次世代軸を経験的に導出する。

References

- [1] Hewitt, C., Bishop, P., and Steiger, R. (1973). *A Universal Modular ACTOR Formalism for Artificial Intelligence*. IJCAI-73.
- [2] Yonezawa, A., Shibayama, E., Takada, T., and Honda, Y. (1986). *Modelling and Programming in an Object-Oriented Concurrent Language ABCL/1*. Object-Oriented Concurrent Programming, MIT Press.
- [3] Hoare, C.A.R. (1978). *Communicating Sequential Processes*. CACM 21(8).
- [4] Lucassen, J.M. and Gifford, D.K. (1988). *Polymorphic Effect Systems*. POPL.
- [5] Wadler, P. (1990). *Linear Types Can Change the World!*. Programming Concepts and Methods.
- [6] Clarke, D. and Drossopoulou, S. (2003). *Ownership, Encapsulation and the Disjointness of Type and Effect*. OOPSLA.
- [7] Honda, K., Vasconcelos, V.T., and Kubo, M. (1998). *Language Primitives and Type Discipline for Structured Communication-Based Programming*. ESOP.
- [8] Armstrong, J. (2007). *Programming Erlang: Software for a Concurrent World*. Pragmatic Bookshelf.
- [9] Lightbend (2010–). *Akka Documentation*. <https://akka.io/docs/>.
- [10] Clebsch, S., Drossopoulou, S., Blessing, S., and McNeil, A. (2015). *Deny Capabilities for Safe, Fast Actors*. AGERE!.
- [11] Pressler, R. (2023). *JEP 444: Virtual Threads*. OpenJDK. <https://openjdk.org/jeps/444>.

本稿で記述した実装は <https://github.com/yaskodama/aio-s-claude> で公開されている。スモークテストの最新結果と機能比較表は同リポジトリの docs/AIPL_Runtime_Feature_Matrix.{md,tex,pdf} に随時更新される。